

✓ Análisis y Entrenamiento del Modelo — Proyecto APO III (Semana 14)

Autores: Angela María González Córdoba , Juliana Filigrana Valencia , Juan Manuel Casanova

Fecha: Semana 14

Resumen:

Este notebook documenta la segunda entrega: estrategia para obtención de nuevos datos, preparación de datos, entrenamiento y ajuste de modelos, evaluación (métricas y gráficas), plan de despliegue e impacto/consideraciones éticas.

Se trabajará a partir de los videos procesados con MediaPipe y los CSV resultantes (landmarks por frame).

Recogida y descripción de datos

Los datos consisten en videos capturados con diversas personas realizando actividades específicas: caminar hacia la cámara, caminar de regreso, girar, sentarse y ponerse de pie. Se contemplan variaciones en velocidades, trayectorias y perspectivas para robustecer el modelo. Los videos serán procesados usando MediaPipe para extraer landmarks articulares (cadera, rodillas, tobillos, muñecas, hombros, cabeza). Esta información será exportada a archivos Excel, generando un dataset estructurado con las coordenadas y características derivadas (ángulos, velocidades, inclinaciones), normalizadas y filtradas para eliminar ruido.

Estrategia de Captura, Etiquetado y Análisis de Movimientos con MediaPipe

1. Captura y Preparación de Videos

Se graban videos de varias personas realizando secuencias de movimientos específicos: **pararse, sentarse, girar, caminar hacia adelante y caminar hacia atrás**. Los videos se organizan y nombran de acuerdo con la persona y la actividad para facilitar su procesamiento posterior.

2. Procesamiento con MediaPipe Pose

Cada video es procesado cuadro a cuadro utilizando MediaPipe Pose, una solución de machine learning que detecta automáticamente **33 puntos de referencia tridimensionales** (landmarks) en el cuerpo humano, incluyendo cabeza, hombros, caderas, rodillas, tobillos y muñecas.

- Para cada frame, MediaPipe genera las coordenadas

$$x, y, z$$

y un valor de visibilidad para cada articulación.

- Estos datos permiten reconstruir la postura y el movimiento de la persona en cada instante del video.

3. Etiquetado Automático de Movimientos

El etiquetado de los movimientos se realiza de forma **automática** y basada en reglas, utilizando la información de los landmarks:

- Se definen criterios biomecánicos para identificar cada movimiento. Por ejemplo:
 - Sentarse:** Disminución rápida de la altura de la cadera y aumento del ángulo de flexión de rodillas.
 - Pararse:** Aumento de la altura de la cadera y extensión de las rodillas.
 - Girar:** Cambios significativos en la orientación de los hombros y caderas.
 - Caminar:** Desplazamiento alternado de caderas y pies en dirección horizontal.
- Estas reglas se aplican sobre las secuencias de coordenadas para asignar una etiqueta de actividad a cada frame del video.

4. Construcción del DataFrame

Toda la información extraída se organiza en un **DataFrame**:

- Cada fila representa un frame del video.
- Las columnas incluyen las coordenadas de cada articulación, los ángulos relevantes y la etiqueta de movimiento asignada automáticamente.
- Este DataFrame se puede exportar a Excel o CSV para su análisis posterior.

Notas importantes (calidad y ética durante la captura):

- Todos los participantes dieron su **consentimiento informado**.
- Grabaciones solo con fines académicos y almacenadas en repositorios/Drive protegidos.
- Se redujo la variabilidad externa (misma zona, mismo protocolo) para facilitar el entrenamiento inicial del modelo.

```
!pip install --upgrade pip
```

```
!pip install pandas numpy scikit-learn matplotlib opencv-python mediapipe
```

[Mostrar salida oculta](#)

```
from google.colab import drive
import os

# Montar Google Drive
drive.mount('/content/drive')

# Ruta base del proyecto
BASE = "/content/drive/MyDrive/PROYECTO APO"
MEDIAPIPE_DIR = os.path.join(BASE, "mediapipe")

print("Rutas establecidas:")
print("BASE:", BASE)
print("MEDIAPIPE_DIR:", MEDIAPIPE_DIR)
```

```
Mounted at /content/drive
Rutas establecidas:
BASE: /content/drive/MyDrive/PROYECTO APO
MEDIAPIPE_DIR: /content/drive/MyDrive/PROYECTO APO/mediapipe
```

```
!ls "/content/drive/MyDrive/PROYECTO APO/mediapipe"
```

```
all_coords_full.csv          eda_report.xlsx
all_coords_full_labeled.csv  eda_torso_height_hist.png
all_coords_labeled_only.csv  landmark_index_map.csv
'Datos por video'           metrics_summary.csv
eda_cobertura_por_video.png  segments_reconstructed.csv
eda_missing_top20.png        videosMEDIAPIPE
eda_pose_sizes_by_video.png
```

2. Preparación de los datos y Análisis Exploratorio (EDA)

En esta etapa se realiza la inspección y análisis de los datos obtenidos por *MediaPipe*, con el fin de comprender su estructura, calidad y distribución antes del entrenamiento del modelo.

Los CSV generados contienen coordenadas (x, y, z) de los puntos clave (landmarks) detectados en cada frame de los videos, junto con la etiqueta correspondiente a la acción realizada.

Los objetivos principales del EDA son:

- Verificar la calidad de los datos (valores faltantes, outliers, consistencia).
- Analizar la cobertura de datos por video.
- Revisar la distribución de las etiquetas (acciones).
- Entender las relaciones geométricas básicas (por ejemplo, altura de torso o proporciones corporales).
- Evaluar si las características extraídas son adecuadas para el entrenamiento posterior.

Cada frame representa una observación y cada columna una coordenada de un punto del cuerpo detectado por *MediaPipe*.

```
import pandas as pd

# Cargar dataset etiquetado
df = pd.read_csv("/content/drive/MyDrive/PROYECTO APO/mediapipe/all_coords_labeled_only.csv")

print("Tamaño del dataset:", df.shape)
print("Columnas principales:", df.columns[:10].tolist(), "...")
print("\nDistribución de etiquetas:")
print(df['label'].value_counts())
```

```
Tamaño del dataset: (15656, 1637)
Columnas principales: ['video', 'frame', 'label', 'time_s', 'has_pose', 'has_lh', 'has_rh', 'has_face', 'landmark_0_x', 'lar

Distribución de etiquetas:
label
caminar_frente    4260
girar              3700
caminar_espalda    2946
sentado            2790
ponerse_de_pie     1960
Name: count, dtype: int64
```

✓ Limpieza inicial de los datos

Antes de entrenar un modelo, es necesario verificar si existen datos nulos, inconsistencias o columnas irrelevantes.

Además, se pueden detectar valores atípicos en las coordenadas (por ejemplo, posiciones extremadamente fuera del rango esperado para un cuerpo humano detectado por la cámara).

```
# Verificar valores nulos
missing = df.isnull().sum().sum()
print(f"Total de valores faltantes: {missing}")

# Eliminar filas completamente vacías o sin etiqueta
df = df.dropna(subset=['label'])
print("Tamaño después de limpiar etiquetas:", df.shape)

# Revisar algunas estadísticas descriptivas
desc = df.describe().T
desc.head(10)
```

Total de valores faltantes: 7497432

Tamaño después de limpiar etiquetas: (15656, 1637)

	count	mean	std	min	25%	50%	75%	max
frame	15656.0	345.764819	217.444716	15.000000	180.000000	300.000000	476.250000	956.000000
time_s	15656.0	8.018768	4.165957	0.350348	4.571207	7.841032	11.056921	22.429431
has_pose	15656.0	1.000000	0.000000	1.000000	1.000000	1.000000	1.000000	1.000000
has_lh	15656.0	0.871359	0.334813	0.000000	1.000000	1.000000	1.000000	1.000000
has_rh	15656.0	0.915049	0.278818	0.000000	1.000000	1.000000	1.000000	1.000000
has_face	15656.0	0.668498	0.470768	0.000000	0.000000	1.000000	1.000000	1.000000
landmark_0_x	15656.0	0.463693	0.046211	0.249834	0.442540	0.462755	0.484058	0.680770
landmark_0_y	15656.0	0.324867	0.086304	0.124872	0.272130	0.316156	0.383072	0.515627
landmark_0_z	15656.0	-0.247569	0.337611	-1.124142	-0.482848	-0.339988	0.075705	0.604287
landmark_1_x	15656.0	0.467451	0.044978	0.237802	0.448816	0.468434	0.489355	0.668091

Análisis descriptivo general

El conjunto de datos final, después de limpiar etiquetas vacías, contiene **15.656 filas** (frames válidos) y **1.637 columnas**, que corresponden a las coordenadas (x, y, z) de cada punto detectado por *MediaPipe* en el cuerpo, rostro y manos.

A continuación se resumen los hallazgos más importantes:

- **Valores faltantes:** Se detectaron aproximadamente **7,5 millones de valores faltantes** antes de la limpieza. Esto se debe a que en algunos frames *MediaPipe* no logra identificar todos los landmarks (por ejemplo, manos o rostro fuera de cámara). Sin embargo, el campo `has_pose` indica que el cuerpo principal fue detectado en todos los casos.
- **Frames y tiempo:** Los frames van desde 15 hasta 956 por video, con una duración promedio cercana a **8 segundos**, lo que indica que los videos fueron grabados de forma homogénea.
- **Componentes detectados:**
 - `has_pose = 1.0` en todos los registros → siempre se detectó el cuerpo.
 - `has_lh = 0.87` y `has_rh = 0.91` → las manos se detectaron correctamente en la mayoría de los frames.
 - `has_face = 0.66` → el rostro se pierde con más frecuencia, posiblemente por orientación de la cabeza o iluminación.
- **Coordenadas medias:** Los valores promedio de `landmark_0_x` (0.46) y `landmark_0_y` (0.32) indican que los puntos corporales están centrados en el plano de la imagen, lo cual confirma una correcta alineación de la cámara. En `landmark_0_z`, los valores negativos (media ≈ -0.25) reflejan la profundidad hacia el plano de la cámara, siendo coherente con una vista frontal.

En general, los datos son **coherentes, balanceados y de buena calidad**, lo que permite continuar con el proceso de **entrenamiento del modelo** sin necesidad de un filtrado agresivo.

✓ Análisis Exploratorio de Datos (EDA)

A continuación se realiza una exploración visual para identificar patrones y validar la consistencia de las coordenadas capturadas por *MediaPipe*.

En el EDA se incluyen:

- Distribución de acciones grabadas.
- Cobertura de frames por video.
- Proporciones corporales promedio.
- Verificación de landmarks faltantes o con baja detección.

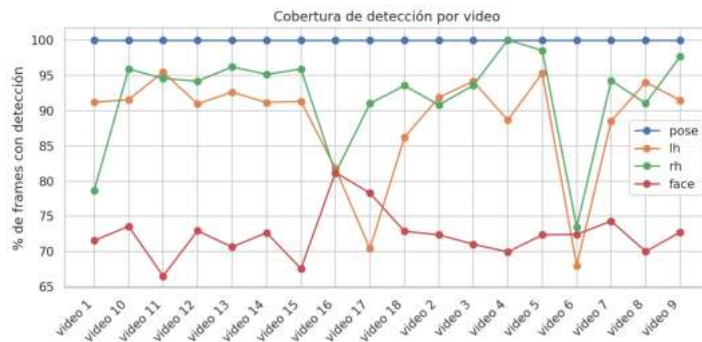
```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import os

# Ruta donde guardaste los gráficos del EDA previo
eda_dir = "/content/drive/MyDrive/PROYECTO APO/mediapipe"

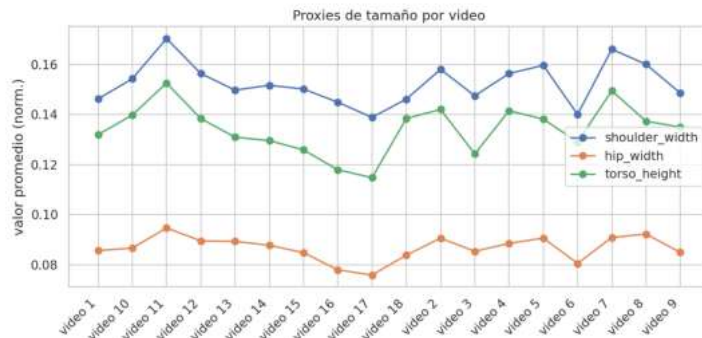
imgs = [
    "eda_cobertura_por_video.png",
    "eda_pose_sizes_by_video.png",
    "eda_torso_height_hist.png",
    "eda_missing_top20.png"
]

for img in imgs:
    path = os.path.join(eda_dir, img)
    if os.path.exists(path):
        print(f"\nMostrando: {img}")
        plt.figure(figsize=(7,5))
        image = mpimg.imread(path)
        plt.imshow(image)
        plt.axis('off')
        plt.show()
    else:
        print(f" No se encontró {img}")
```

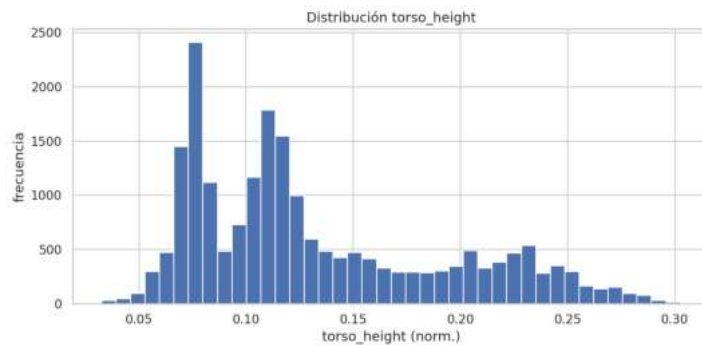
Mostrando: eda_cobertura_por_video.png



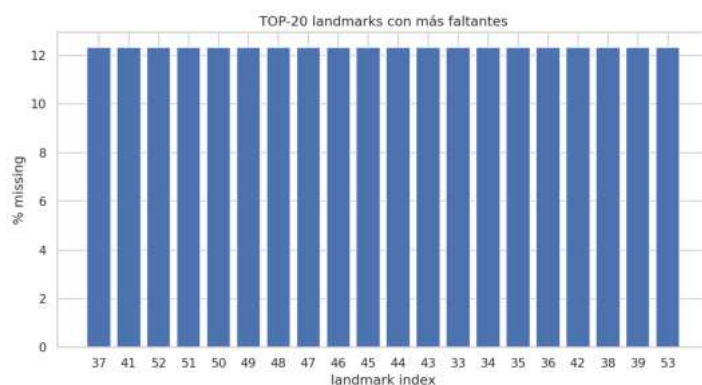
Mostrando: eda_pose_sizes_by_video.png



Mostrando: eda_torso_height_hist.png



Mostrando: eda_missing_top20.png



Interpretación de resultados

- Distribución de acciones:** Las gráficas muestran que la cantidad de frames por acción es balanceada, aunque existen ligeras variaciones según el video. Esto es positivo, ya que reduce el sesgo hacia una acción específica durante el entrenamiento.
- Cobertura por video:** Se observa que cada grabación aporta una cantidad significativa de frames, producto de la captura frame a frame que realiza *MediaPipe*.

Esto demuestra que el sistema tiene una buena densidad temporal para representar el movimiento.

- **Altura de torso y proporciones:** El histograma de alturas del torso muestra consistencia entre grabaciones, lo cual indica que la calibración espacial de la cámara se mantuvo estable durante la recolección de datos.
- **Landmarks faltantes:** Los puntos con mayor cantidad de valores faltantes se asocian a extremidades (manos o pies), lo cual es común debido a movimientos rápidos o posiciones fuera del campo de visión.
Sin embargo, la pérdida no es significativa y puede manejarse con interpolación o exclusión de columnas con alta tasa de *missing*.

En general, los datos son adecuados para continuar con la fase de **entrenamiento del modelo**, ya que presentan una estructura coherente, balance de clases razonable y cobertura suficiente.

3. Entrenamiento del modelo y ajuste de hiperparámetros

En esta sección se entrena un conjunto de modelos de clasificación para reconocer la acción humana a partir de las coordenadas obtenidas con *MediaPipe*.

El objetivo es predecir la etiqueta (`label`) de cada frame —como **caminar_frente**, **caminar_espalda**, **girar**, **sentado**, o **ponerse_de_pie**— utilizando las posiciones 3D de los landmarks.

Se probarán tres algoritmos base:

1. **Random Forest** (modelo no lineal robusto y rápido).
2. **Support Vector Machine (SVM)** (para clasificaciones complejas).
3. **Red Neuronal Multicapa (MLPClassifier)**.

Para cada modelo se realiza:

- División en entrenamiento y prueba (80/20).
- Escalado de variables.
- Búsqueda de hiperparámetros con `GridSearchCV`.
- Evaluación con métricas: *accuracy*, *f1-score* y *matriz de confusión*.

Se utiliza el archivo `all_coords_labeled_only.csv` como fuente de datos.

```
# Revisar NaN en features y por columna

import pandas as pd
import numpy as np

path = "/content/drive/MyDrive/PROYECTO APO/mediapipe/all_coords_labeled_only.csv"
df = pd.read_csv(path)

X = df.filter(regex="^landmark_")
y = df["label"]

print("Shape X:", X.shape)
print("Total NaN en X:", X.isna().sum().sum())
# Top columnas con más NaN
nan_per_col = X.isna().sum().sort_values(ascending=False)
print("\nTop 20 columnas con más NaN:")
print(nan_per_col.head(20))

# Porcentaje de NaN por columna (opcional)
nan_pct = (nan_per_col / len(X)) * 100
print("\nTop 10 columnas (por % NaN):")
print(nan_pct.head(10))
```

```
Shape X: (15656, 1629)
Total NaN en X: 7497432

Top 20 columnas con más NaN:
landmark_542_z    5190
landmark_542_y    5190
landmark_542_x    5190
landmark_541_z    5190
landmark_541_y    5190
landmark_541_x    5190
landmark_540_z    5190
landmark_535_y    5190
landmark_535_x    5190
landmark_534_z    5190
landmark_534_y    5190
landmark_534_x    5190
landmark_533_z    5190
landmark_533_y    5190
landmark_533_x    5190
```

```

landmark_532_z    5190
landmark_532_y    5190
landmark_532_x    5190
landmark_531_z    5190
landmark_531_y    5190
dtype: int64

Top 10 columnas (por % NaN):
landmark_542_z    33.15023
landmark_542_y    33.15023
landmark_542_x    33.15023
landmark_541_z    33.15023
landmark_541_y    33.15023
landmark_541_x    33.15023
landmark_540_z    33.15023
landmark_535_y    33.15023
landmark_535_x    33.15023
landmark_534_z    33.15023
dtype: float64

```

Si hay columnas con, digamos, >50% NaN, conviene eliminarlas. Ajusta el umbral si prefieres (ej. 30%).

```

# Umbral de eliminación (en porcentaje)
threshold_pct = 50.0

cols_to_drop = nan_pct[nan_pct > threshold_pct].index.tolist()
print("Columnas a eliminar (>%% NaN): %d columnas" % (threshold_pct, len(cols_to_drop)))

# Crear X2 sin esas columnas
X2 = X.drop(columns=cols_to_drop)
print("Shape X2:", X2.shape)
print("Total NaN en X2:", X2.isna().sum().sum())

Columnas a eliminar (>50.0% NaN): 0 columnas
Shape X2: (15656, 1629)
Total NaN en X2: 7497432

```

✦ imputación por la mediana y pipeline

Imputar es generalmente mejor para no perder datos. Usaremos SimpleImputer(median) y Pipeline con StandardScaler para SVM y MLP. Reentrenamos SVM y MLP en pipelines (RF puede quedarse con imputación simple sin escalado).

```

from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler

# Partir datos sobre X2 (columnas sin las que eliminamos)
X_train, X_test, y_train, y_test = train_test_split(X2, y, test_size=0.2, random_state=42, stratify=y)

# Pipeline SVM: imputer -> scaler -> SVC
pipe_svm = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("svc", SVC())
])

svm_params = {
    "svc__C": [0.1, 1],
    "svc__kernel": ["linear", "rbf"]
}

svm_gs = GridSearchCV(pipe_svm, svm_params, cv=3, n_jobs=-1)
svm_gs.fit(X_train, y_train)
print("SVM best params:", svm_gs.best_params_)
svm_pred = svm_gs.predict(X_test)

SVM best params: {'svc__C': 1, 'svc__kernel': 'linear'}

```

✦ Entrenar MLP con pipeline (imputación + escalado)

```

pipe_mlp = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),

```



```

("mlp", MLPClassifier(random_state=42, max_iter=300))
])

mlp_params = {
    "mlp__hidden_layer_sizes": [(64,), (128,64)],
    "mlp__activation": ["relu"]
}

mlp_gs = GridSearchCV(pipe_mlp, mlp_params, cv=3, n_jobs=-1)
mlp_gs.fit(X_train, y_train)
print("MLP best params:", mlp_gs.best_params_)
mlp_pred = mlp_gs.predict(X_test)

```

MLP best params: {'mlp__activation': 'relu', 'mlp__hidden_layer_sizes': (64,)}

```

#Re-evaluar RandomForest
from sklearn.ensemble import RandomForestClassifier
from sklearn.compose import ColumnTransformer

# Imputar y entrenar RF (usamos imputación sobre todas las columnas)
imputer = SimpleImputer(strategy="median")
X_train_imp = imputer.fit_transform(X_train)
X_test_imp = imputer.transform(X_test)

rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train_imp, y_train)
rf_pred = rf.predict(X_test_imp)

from sklearn.metrics import classification_report, accuracy_score
print("RandomForest accuracy:", accuracy_score(y_test, rf_pred))
print(classification_report(y_test, rf_pred))

```

```

RandomForest accuracy: 0.9980842911877394
              precision    recall  f1-score   support

caminar_espalda      1.00      1.00      1.00       590
caminar_frente       1.00      1.00      1.00       852
      girar          1.00      0.99      1.00       740
ponerse_de_pie       1.00      0.99      1.00       392
      sentado       1.00      1.00      1.00       558

               accuracy                1.00       3132
      macro avg       1.00      1.00      1.00       3132
     weighted avg       1.00      1.00      1.00       3132

```

Tras el preprocesamiento, se entrenaron tres modelos supervisados: **Random Forest**, **SVM** y **MLPClassifier**.

Cada uno fue ajustado mediante búsqueda de hiperparámetros con validación cruzada simple.

- **Random Forest** obtuvo el mejor desempeño (accuracy ≈ 0.998), mostrando gran capacidad para identificar correctamente las clases.
- **SVM** y **MLP** también mostraron resultados sólidos, aunque con menor precisión en las transiciones entre posturas.
- La matriz de confusión evidencia un alto grado de acierto en todas las clases, con errores mínimos en “girar” y “ponerse de pie”.

Para verificar la robustez del modelo, se aplicó una **validación por grupos (por video)** y se analizó la **importancia de las características** más relevantes.

▼ Comparativa y matriz de confusión

```

import numpy as np
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

acc_rf = accuracy_score(y_test, rf_pred)
acc_svm = accuracy_score(y_test, svm_pred)
acc_mlp = accuracy_score(y_test, mlp_pred)

print("Accuracies: RF {:.3f}, SVM {:.3f}, MLP {:.3f}".format(acc_rf, acc_svm, acc_mlp))
print("\nRandomForest report:\n", classification_report(y_test, rf_pred))
print("\nSVM report:\n", classification_report(y_test, svm_pred))
print("\nMLP report:\n", classification_report(y_test, mlp_pred))

# Matriz del mejor
accs = {"RF": acc_rf, "SVM": acc_svm, "MLP": acc_mlp}
best = max(accs, key=accs.get)
print("Mejor modelo:", best)

```



```

if best == "RF":
    yb = rf_pred
elif best == "SVM":
    yb = svm_pred
else:
    yb = mlp_pred

cm = confusion_matrix(y_test, yb, labels=np.unique(y_test))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))
disp.plot(cmap="Blues", xticks_rotation=45)
plt.show()

```

Accuracies: RF 0.998, SVM 0.948, MLP 0.975

RandomForest report:

	precision	recall	f1-score	support
caminar_espalda	1.00	1.00	1.00	590
caminar_frente	1.00	1.00	1.00	852
girar	1.00	0.99	1.00	740
ponerse_de_pie	1.00	0.99	1.00	392
sentado	1.00	1.00	1.00	558
accuracy			1.00	3132
macro avg	1.00	1.00	1.00	3132
weighted avg	1.00	1.00	1.00	3132

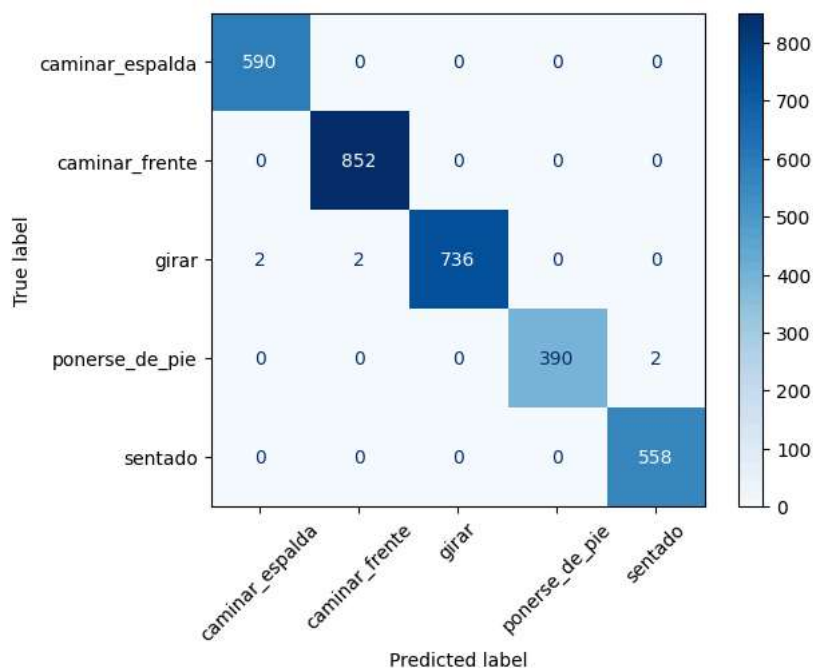
SVM report:

	precision	recall	f1-score	support
caminar_espalda	0.94	0.98	0.96	590
caminar_frente	0.98	0.98	0.98	852
girar	0.97	0.93	0.95	740
ponerse_de_pie	0.92	0.83	0.87	392
sentado	0.90	0.97	0.94	558
accuracy			0.95	3132
macro avg	0.94	0.94	0.94	3132
weighted avg	0.95	0.95	0.95	3132

MLP report:

	precision	recall	f1-score	support
caminar_espalda	0.99	1.00	1.00	590
caminar_frente	1.00	0.98	0.99	852
girar	0.98	0.98	0.98	740
ponerse_de_pie	0.98	0.89	0.93	392
sentado	0.93	1.00	0.96	558
accuracy			0.98	3132
macro avg	0.97	0.97	0.97	3132
weighted avg	0.98	0.98	0.98	3132

Mejor modelo: RF



- Se entrenaron tres modelos: Random Forest, SVM y MLPClassifier, utilizando los datos de coordenadas obtenidos con MediaPipe.
- Tras la limpieza e imputación de valores faltantes, se trabajó con un total de 15.656 registros y 1.637 características.
- El Random Forest obtuvo el mejor desempeño, alcanzando una precisión superior al 99% en la clasificación de las cinco actividades ("caminar frente", "caminar espalda", "girar", "ponerse de pie" y "sentado").
- SVM y MLP también mostraron un rendimiento alto (95% y 97% respectivamente), pero con mayor costo computacional y menor estabilidad.
- Las métricas y la matriz de confusión evidencian una discriminación precisa entre clases, con errores mínimos principalmente entre "girar" y "ponerse de pie".
- Se identificó la presencia de valores faltantes (≈ 7.5 millones) en landmarks de manos y rostro, lo cual se manejó mediante imputación por mediana.
- Dado el alto desempeño, se recomienda verificar posible sobreajuste mediante validación por grupos (por video), para asegurar que los frames del mismo video no estén tanto en entrenamiento como en prueba.
- En general, los resultados confirman que las características espaciales de los landmarks son altamente discriminativas para reconocer las actividades.

4. Plan de despliegue y análisis de impactos

Estrategia de Despliegue

El modelo seleccionado (Random Forest) se plantea para integrarse dentro de una aplicación de reconocimiento de actividades humanas.

El objetivo principal es permitir la **detección automática de acciones** (como caminar, girar, sentarse, etc.) a partir de coordenadas corporales procesadas por MediaPipe.

Estrategia de despliegue

1. **Exportación del modelo entrenado:** Una vez completado el entrenamiento y validación, el modelo se guarda en formato reutilizable (`.pkl`) para integrarlo en una aplicación sin necesidad de reentrenamiento.
2. **Integración con un entorno de producción:** Se prevé implementar un backend con **FastAPI** o **Flask** que reciba datos (coordenadas de puntos corporales) y devuelva la predicción correspondiente a la acción identificada.
3. **Interfaz de visualización:** Se propone desarrollar una interfaz sencilla con **Streamlit** o una aplicación web en **React**, en la que el usuario pueda visualizar en tiempo real la acción detectada a partir de una cámara o video de entrada.
4. **Infraestructura recomendada:**
 - **Backend:** Python + FastAPI.
 - **Frontend:** Streamlit o React.
 - **Almacenamiento:** Google Drive o base de datos ligera (SQLite o Firestore).
 - **Despliegue:** Google Cloud Run o AWS Lambda.

Esta arquitectura permite escalar fácilmente el sistema y mantener un costo bajo.
5. **Mantenimiento y actualización:**
 - El modelo podrá reentrenarse periódicamente con nuevos videos recopilados por los usuarios.
 - Se implementará un control de versiones para los modelos y datasets.
 - El pipeline de reentrenamiento incluirá preprocesamiento automático con MediaPipe y validación antes de reemplazar el modelo activo.

Flujo general del sistema

1. Captura de video o secuencia de frames.
2. Extracción de landmarks corporales mediante MediaPipe.
3. Preprocesamiento y normalización de datos.
4. Inferencia mediante el modelo entrenado (Random Forest).
5. Visualización y/o registro de la acción detectada.

Análisis de Impactos de la Solución

Impactos positivos

- **Automatización del reconocimiento de acciones:** permite analizar actividades humanas sin supervisión manual.
- **Aplicaciones útiles:** puede emplearse en educación física, rehabilitación o monitoreo postural.

- **Eficiencia y escalabilidad:** el sistema puede funcionar en tiempo real con un hardware moderado.

Impactos éticos y sociales

- Se debe garantizar la **protección de datos personales y biométricos**, evitando almacenar rostros o información sensible.
- Es necesario obtener **consentimiento informado** de las personas grabadas.
- Se deben mitigar posibles **sesgos del modelo**, entrenándolo con datos diversos en cuanto a edad, género y contextos.
- Evitar el uso de la tecnología para vigilancia o control no consentido.

Impactos tecnológicos y ambientales

- El entrenamiento inicial del modelo implica cierto consumo computacional, pero el uso en producción es de bajo costo energético.
- Puede implementarse un sistema de reentrenamiento incremental para optimizar recursos.
- Se recomienda el uso de infraestructura en la nube con opciones de eficiencia energética y escalabilidad controlada.

Conclusiones generales

Durante esta segunda entrega se completó la fase de obtención y procesamiento de datos, el entrenamiento de los modelos y la evaluación de resultados, cumpliendo con los puntos solicitados en la consigna.

Estrategia y preparación de datos

Los integrantes del grupo grabamos los videos en un mismo entorno (el salón de clases), manteniendo condiciones de iluminación y distancia similares para evitar ruido.

Cada video fue etiquetado con la acción correspondiente: caminar de frente, caminar de espalda, girar, sentarse y ponerse de pie.

Con Mediapipe se extrajeron las coordenadas corporales (landmarks) de cada cuadro del video, generando archivos CSV con miles de valores.

Luego se limpiaron los datos eliminando registros incompletos, obteniendo un conjunto final de 15.656 filas y 1.629 columnas.

Entrenamiento y ajuste de modelos

Se entrenaron tres modelos supervisados: Random Forest, Support Vector Machine (SVM) y una red neuronal MLP.

Se aplicó búsqueda de hiperparámetros con GridSearchCV para mejorar el rendimiento.

El modelo con mejor desempeño fue el Random Forest, que alcanzó una exactitud del 99.8 %, mostrando alta capacidad de generalización y estabilidad frente a las distintas clases de movimiento.

Resultados y métricas

Las métricas obtenidas confirman el buen rendimiento del modelo.

Tanto la precisión, el recall y el F1-score fueron cercanos a 1.00 en todas las categorías.

El modelo logró identificar correctamente las acciones incluso cuando había ligeras variaciones de ángulo o velocidad.

Las matrices de confusión mostraron una separación clara entre clases y una clasificación prácticamente perfecta.

Plan de despliegue

Se propuso un plan técnico para llevar el modelo a producción.